

CHƯƠNG 6. DUYỆT ĐỒ THỊ

Trong chương 6 sẽ khảo sát các phương pháp duyệt đồ thị. Các phương pháp này sẽ được ứng dụng để giải quyết các bài toán khác nhau trên đồ thị. Những bài toán được xem xét trực tiếp ở đây là bài toán tìm đường đi và kiểm tra tính liên thông của đồ thị.

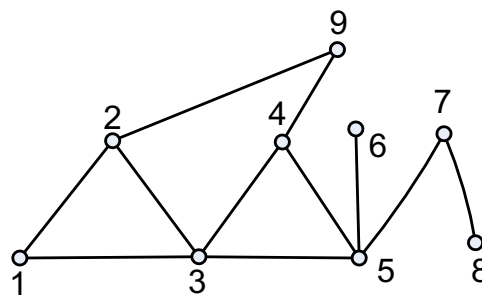
Duyệt đồ thị là quá trình đi qua tất cả các đỉnh của đồ thị sao cho mỗi đỉnh của nó được viếng thăm đúng một lần. Nội dung của giáo trình đề cập hai phương pháp duyệt đồ thị: Duyệt theo chuyên sâu (Depth First Search) và duyệt theo chiều rộng (Breadth First Search).

6.1. DUYỆT ĐỒ THỊ THEO CHIỀU SÂU

6.1.1. Phương pháp duyệt

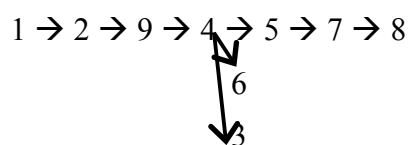
Bắt đầu duyệt từ một đỉnh v nào đó của đồ thị. Tiếp theo, chọn u là một đỉnh tùy ý kề với v (với đồ thị có hướng thì u là đỉnh cuối, v là đỉnh đầu của cung uv) và lặp lại quá trình này với u cho đến khi không tìm được đỉnh kề (chưa được duyệt) tiếp theo nữa thì quay trở lại đỉnh ngay trước đó để tìm qua nhánh khác.

Ví dụ: Thực hiện duyệt chiều sâu đồ thị sau:



Hình 6.1

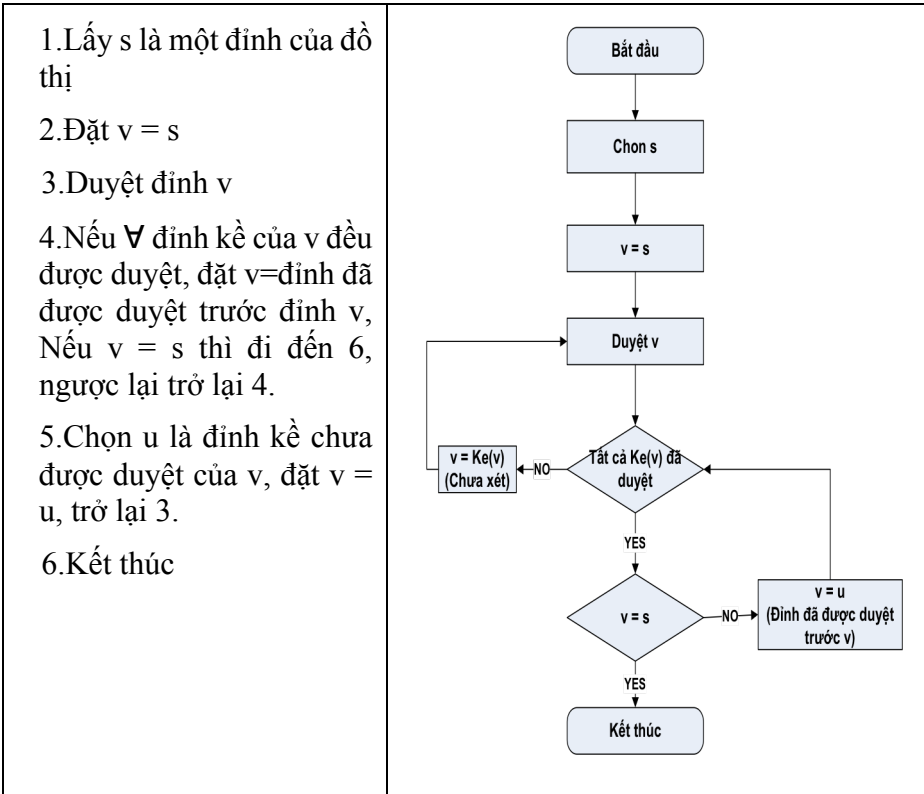
Nếu bắt đầu từ đỉnh 1 thì thứ tự các đỉnh được duyệt theo chiều sâu như sau:



6.1.2. Cài đặt thuật toán

Với nguyên lý duyệt đồ thị như vậy, ta có thể được cài đặt theo hai cách: Đệ quy và không đệ quy (sử dụng cấu trúc dữ liệu ngăn xếp – Stack).

a. Thuật toán đệ quy



Dưới đây là giải thuật duyệt đồ thị theo chiều sâu bằng phương pháp đệ quy được viết bằng mã giả. Trong giải thuật, ta khai báo một mảng $Ke[]$ để lưu danh sách kề của các đỉnh, và mảng $ChuaXet[]$ để đánh dấu một đỉnh nào đó đã được duyệt hay chưa.

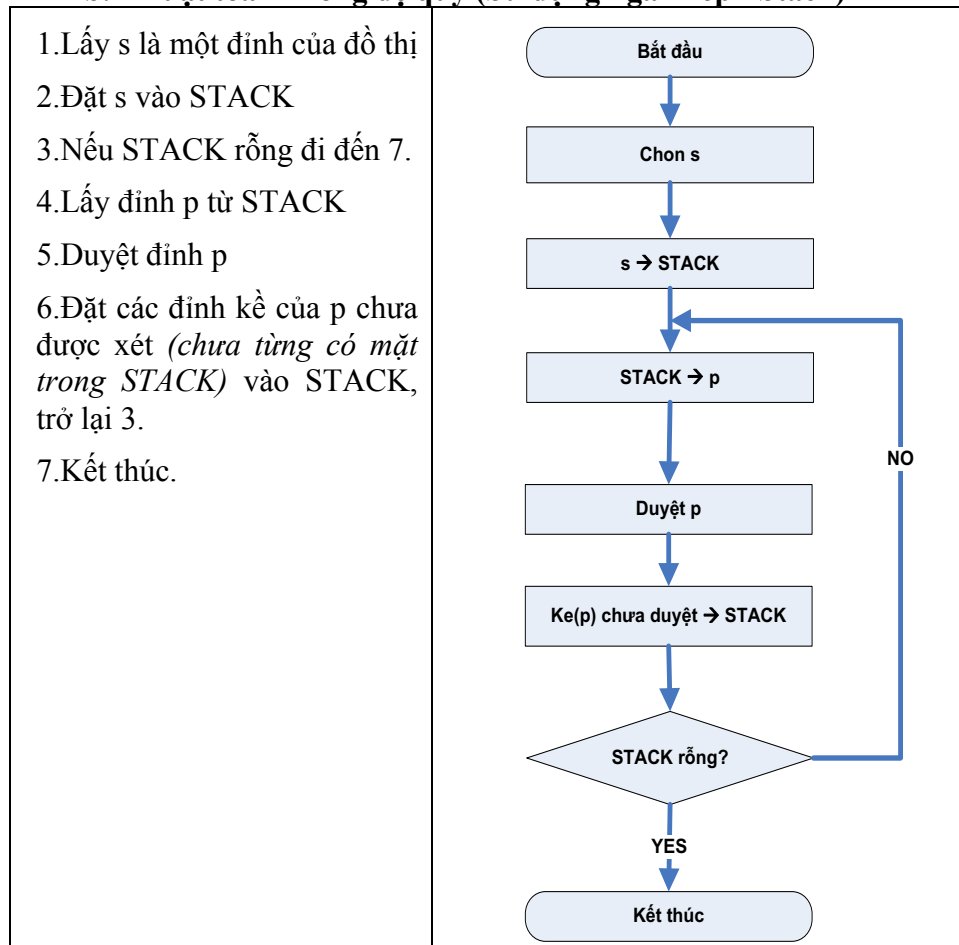
```

/* Khai báo các biến ChuaXet, Ke */
DFS(v)
{
    Duyệt đỉnh (v);
    ChuaXet[v] = 0; /*Đánh dấu đã xét đỉnh v*/
    for (u ∈ Ke(v))
        if (ChuaXet[u]) DFS(u);
};

void main()
{
    /* Nhập đồ thị, tạo mảng Ke */
    for (v ∈ V) ChuaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
    for (v ∈ V)
        if (ChuaXet[v]) DFS(v);
}

```

b. Thuật toán không đệ quy (Sử dụng ngăn xếp - Stack)



Cũng theo ví dụ hình 3-1, quá trình thực thi của giải thuật được minh họa dưới đây.

Bắt đầu duyệt từ đỉnh 1. Nội dung ngăn xếp sẽ thay đổi như sau:

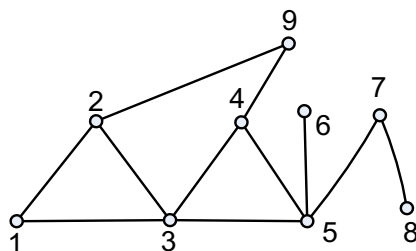
Begin	STACK									End
∅	1	→ 2	→ 9	→ 4	→ 5	↗ 7	→ 8	→ 6	→ 3	∅
		3	3	3	3	6	6	3		
						3	3			

6.2. DUYỆT ĐỒ THỊ THEO CHIỀU RỘNG

6.2.1. Phương pháp duyệt

Bắt đầu từ một đỉnh v bất kỳ, duyệt đỉnh v sau đó lưu tất cả những đỉnh kề với v vào hàng đợi. Tiếp theo, lấy đỉnh u ở đầu hàng đợi, duyệt u và làm tương tự giống như với v.

Ví dụ:



Hình 6.2

Nếu bắt đầu duyệt từ đỉnh 1 thì ta có cách duyệt theo chiều rộng như sau:

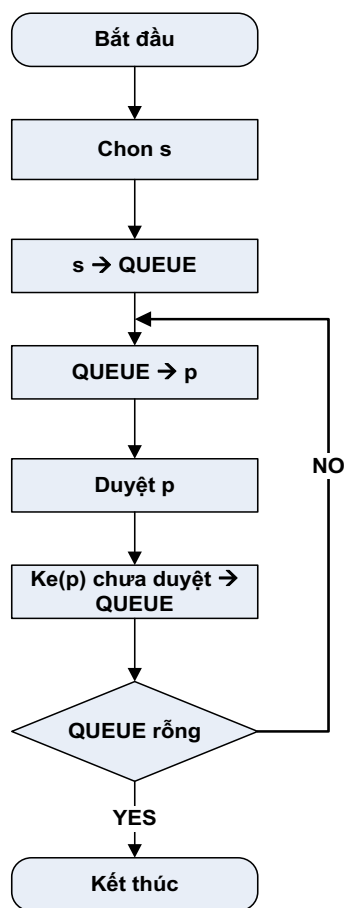
1 2 3 9 4 5 6 7 8

6.2.2. Cài đặt thuật toán

Tương tự như phương pháp duyệt đồ thị theo chiều sâu, phương pháp duyệt theo chiều rộng có thể được cài đặt theo hai cách: đệ quy và không đệ quy (sử dụng cấu trúc dữ liệu hàng đợi - Queue)

a. Thuật toán không đệ quy (Sử dụng hàng đợi – Queue)

1. Lấy s là một đỉnh của đồ thị
2. Đặt s vào QUEUE
3. Lặp nếu QUEUE chưa rỗng.
4. Lấy đỉnh p từ QUEUE
5. Duyệt đỉnh p
6. Đặt các đỉnh kề của p chưa được xét (*chưa từng có mặt trong QUEUE*) vào QUEUE.
7. Kết thúc



Chúng ta dễ dàng nhận thấy, giải thuật duyệt đồ thị theo chiều rộng không đệ quy hoàn toàn tương tự với giải thuật duyệt đồ thị theo chiều sâu không đệ quy. Điểm khác biệt là ta thay thế kiểu dữ liệu ngăn xếp bằng kiểu dữ liệu hàng đợi.

Dưới đây là minh họa quá trình thực thi của giải thuật cho ví dụ ở (Hình 3.2).

Bắt đầu duyệt từ đỉnh 1, nội dung hàng đợi sẽ thay đổi như sau:

Begin	QUEUE								End	
∅	1	2	3	9	4	5	6	7	8	∅
		3	9	4	5		7			
				5						

Dưới đây là phần cài đặt bằng mã giả cho thuật toán. Tương tự như thuật toán duyệt theo chiều sâu, ta cần khai báo một mảng Ke[] và mảng ChuaXet[]. Ngoài ra, ta cần xây dựng một cấu trúc QUEUE để sử dụng cho thuật toán.

```

/* Khai báo các biến ChuaXet, Ke */
BFS(v)
{
    QUEUE = ∅;
    QUEUE ← v;
    ChuaXet[v] = 0; /*Đánh dấu đã xét đỉnh v*/
    while (QUEUE ≠ ∅)
    {
        p ← QUEUE;
        Duyệt đỉnh p;
        for (u ∈ Ke(p))
            if (ChuaXet[u])
            {
                QUEUE ← u;
                ChuaXet[u] = 0; /*Đánh dấu đã xét đỉnh */
            }
    }
}
main()
/* Nhập đồ thị, tạo biến Ke */
{

```

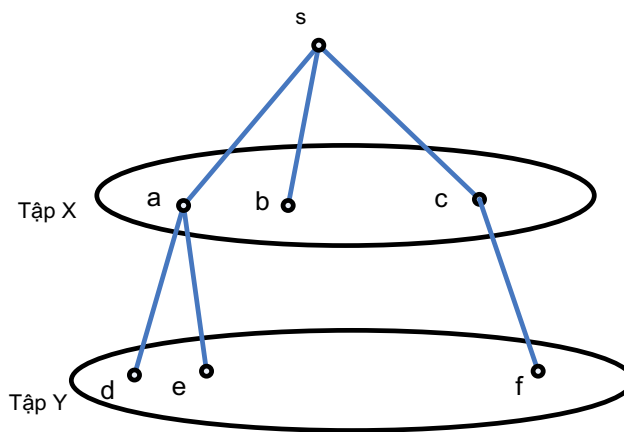
```

for (v ∈ V) ChưaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
for (v ∈ V)
    if (ChưaXet[v]) BFS(v);
}

```

b. Thuật toán đệ quy

Thuật toán duyệt đồ thị theo chiều rộng đệ quy còn gọi là thuật toán loang. Ta sử dụng hai tập hợp X, Y để lưu trữ thông tin các đỉnh được duyệt theo cùng cấp. Chẳng hạn, theo hình bên dưới, sau khi đỉnh s được duyệt, các đỉnh tiếp theo được duyệt (xem là cùng cấp) là a, b, c. Tương tự, các đỉnh được duyệt kế tiếp là d, e, f.



Hình 6.3

Thuật toán được trình bày bằng mã giả:

Bước 1: Khởi tạo

- Bắt đầu từ đỉnh s. Đánh dấu đỉnh s, các đỉnh khác s đều chưa bị đánh dấu
- $X = \{s\}$, $Y = \emptyset$

Bước 2: Lặp lại cho đến khi $X = \emptyset$

- Gán $Y = \emptyset$.
- Với mọi đỉnh $u \in X$
 - Xét tất cả các đỉnh v kề với u mà chưa bị đánh dấu. Với mỗi đỉnh đó:
 - Đánh dấu v
 - Lưu đường đi, đỉnh liền trước v trong đường đi từ s $\rightarrow$ v là u.
 - Đưa v vào tập Y
- Gán $X = Y$

6.3. TÌM ĐƯỜNG ĐI VÀ KIỂM TRA TÍNH LIÊN THÔNG

Các thuật toán duyệt đồ thị ở trên có thể áp dụng để giải quyết bài toán tìm đường đi giữa hai đỉnh và kiểm tra tính liên thông của đồ thị.

6.3.1. Bài toán tìm đường đi

Cho đồ thị G , s và t là hai đỉnh tùy ý của đồ thị. Hãy tìm đường đi từ s đến t .

- **Nhận xét**

Như ta đã biết, các hàm duyệt đồ thị DFS(v) và BFS(v) cho phép thăm các đỉnh của đồ thị G . Do đó, nếu ta bắt đầu duyệt từ đỉnh s và sau khi kết thúc thuật toán mà giá trị ChuaXet[t] = 1 thì điều đó có nghĩa là *không có đường đi từ s đến t* , còn nếu ChuaXet[t] = 0 thì có nghĩa là *có đường đi từ s đến t* . Ngoài ra, ta sẽ thêm biến Truoc[] để lưu vết. Sau đây là các giải thuật viết bằng mã giả để tìm đường đi bằng cách duyệt theo chiều sâu, và chiều rộng.

- **Thuật toán tìm đường đi theo chiều sâu**

```
/* Khai báo các biến ChuaXet, Ke */
DFS(v)
{
    Duyệt đỉnh (v);
    ChuaXet[v] = 0;
    for (u ∈ Ke(v))
        if (ChuaXet[u])
        {
            Truoc[u] = v; /* Lưu vết*/
            DFS(u);
        }
}
main()
{ /* Nhập đồ thị, tạo biến Ke */
    for (v ∈ V) ChuaXet[v] = 1; /* Khởi tạo cờ cho đỉnh*/
    /* Nhập các đỉnh s, t */
    DFS(s);
    if(ChuaXet[t])
        /*Xuất “có đường đi từ s đến t”*/
    else
        /* Xuất “Không có đường đi từ s đến t”*/
}
```

- **Thuật toán tìm đường đi theo chiều rộng**

```

/* Khai báo các biến ChuaXet, Ke, QUEUE */
BFS(v);
{
    QUEUE =  $\emptyset$ ;
    QUEUE  $\leftarrow$  v;
    ChuaXet[v] = 0;
    while (QUEUE  $\neq \emptyset$ )
    {
        p  $\leftarrow$  QUEUE;
        Duyệt đỉnh p;
        for (u  $\in$  Ke(p))
            if (ChuaXet[u])
            {
                QUEUE  $\leftarrow$  u;
                ChuaXet[u] = 0;
                Truoc[u] = p; /* Lưu vết */
            }
    }
}

main()
{ /* Nhập đồ thị, tạo biến Ke */
    for (v  $\in$  V) ChuaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
    /* Nhập các đỉnh s, t */
    BFS(s);
    if(ChuaXet[t])
        /* Xuất “có đường đi từ s đến t” */
    else
        /* Xuất “Không có đường đi từ s đến t” */
}

```

6.3.2. Bài toán kiểm tra tính liên thông

Tính số thành phần liên thông của đồ thị, và xác định những đỉnh thuộc cùng một thành phần liên thông.

- **Nhận xét**

Các hàm DFS(s) và BFS(s) cho phép ta duyệt tất cả các đỉnh thuộc cùng một thành phần liên thông với s. Vì vậy, số thành phần liên thông bằng số lần gọi đến hàm này. Trong thuật toán kiểm tra tính liên thông của đồ thị dưới đây, ta thêm biến inconnect để đếm số thành phần liên thông của đồ thị, và biến index[v] để lưu chỉ số của thành phần liên thông chứa đỉnh v. Mỗi khi gọi đến hàm DFS(v) hay BFS(v) sẽ gán $\text{index}[v] = \text{inconnect}$.

- **Thuật toán kiểm tra tính liên thông theo chiều sâu**

```
/* Khai báo các biến ChuaXet, Ke, index */
DFS(v);
{
    Duyệt đỉnh (v);
    index[v] = inconnect;
    ChuaXet[v] = 0;
    for (u ∈ Ke(v))
        if (ChuaXet[u]) DFS(u);
}
main()
{ /* Nhập đồ thị, tạo biến Ke */
    for (v ∈ V) ChuaXet[v] = 1; /* Khởi tạo cờ cho đỉnh */
    inconnect = 0;
    for (v ∈ V)
        if (ChuaXet[v])
        {
            inconnect ++;
            DFS(v);
        }
}
```

- **Thuật toán kiểm tra tính liên thông theo chiều rộng**

```
/* Khai báo các biến toàn cục ChuaXet, Ke, QUEUE, index */
BFS(v);
{
    QUEUE = 0;
```

```

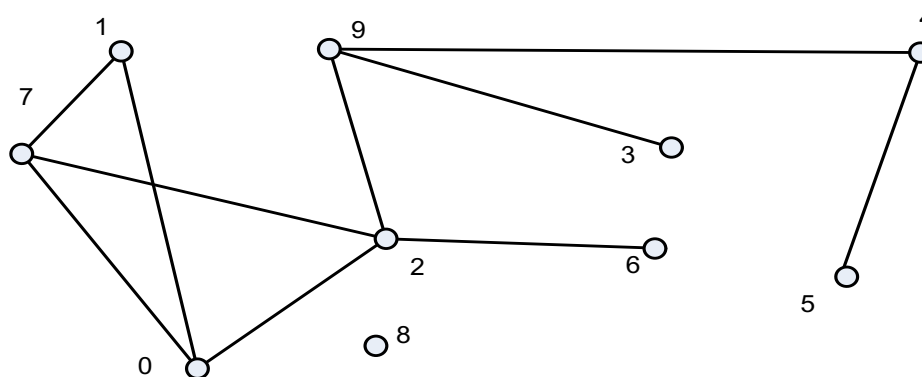
QUEUE  $\leftarrow$  v;
ChuaXet[v] = 0;
while (QUEUE  $\neq$  0) {
    p  $\leftarrow$  QUEUE;
    Duyệt đỉnh p;
    index[p] = inconnect;
    for (u  $\in$  Ke(p))
        if (ChuaXet[u])
            {
                QUEUE  $\leftarrow$  u;
                ChuaXet[u] = 0;
            }
    }
}
main()
{
    for (v  $\in$  V) ChuaXet[v] = 1;
    inconnect = 0;
    for (v  $\in$  V)
        if (ChuaXet[v])
            {
                inconnect ++;
                BFS(v);
            }
}

```

6.4. BÀI TẬP CHƯƠNG 6

1. Cho đồ thị vô hướng, sử dụng thuật toán duyệt đồ thị, hãy trình bày các bước duyệt tất cả các đỉnh của đồ thị sau:

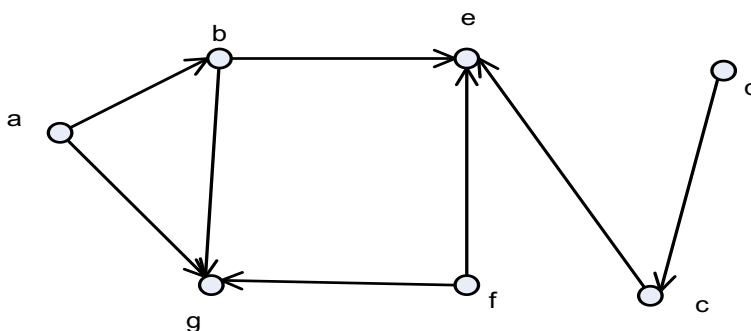
- a) Duyệt theo chiều sâu
- b) Duyệt theo chiều rộng



Hình 6.4

2. Cho đồ thị có hướng, sử dụng thuật toán duyệt đồ thị, hãy trình bày các bước duyệt tất cả các đỉnh của đồ thị sau:

- Duyệt theo chiều sâu
- Duyệt theo chiều rộng



Hình 6.5

3. Viết chương trình thực hiện duyệt đồ thị theo chiều sâu bằng phương pháp đệ quy và không đệ quy.

4. Viết chương trình thực hiện duyệt đồ thị theo chiều rộng bằng phương pháp đệ quy và không đệ quy.

5. Áp dụng hàm duyệt đồ thị theo chiều sâu, hãy tìm tất cả các cạnh là cầu trên đồ thị vô hướng $G = (V, E)$.

6. Áp dụng hàm duyệt đồ thị theo chiều rộng, viết chương trình tìm đường đi giữa hai đỉnh của đồ thị.

7. Áp dụng hàm duyệt đồ thị theo chiều sâu, viết chương trình kiểm tra tính liên thông của đồ thị. Cho biết các đỉnh trong cùng một thành phần liên thông.

8. Sử dụng ngôn ngữ lập trình C, C++ hoặc C#, viết chương trình kiểm tra xem một đồ thị đơn vô hướng có phải là đồ thị vòng hay không.

9. Cho một bảng hình chữ nhật chia thành $M \times N$ ô vuông. Mỗi ô vuông ghi một số nguyên dương (trong khoảng từ 1 đến 255). Một miền của bảng là tập hợp tất cả các ô có cùng giá trị và

có thể đi được sang nhau bằng cách đi qua các ô có chung cạnh. Diện tích của mỗi miền là số ô thuộc miền đó. Hãy viết chương trình đếm số miền của bảng và cho biết diện tích của miền lớn nhất.

Ví dụ:

Bảng kích thước 5×4 . Có 6 miền.

Diện tích miền lớn nhất là: 5.

1	1	3	3
2	1	3	1
2	2	3	1
2	1	1	3
1	1	1	3

10. Một nhóm gồm N lập trình viên được đánh số từ 0 đến $N-1$. Một số người trong họ có địa chỉ email của nhau. Khi biết một thông tin nào mới, họ có thể gửi mail để trao đổi thông tin với nhau. Giả sử bạn có email của tất cả N lập trình viên này, và biết rõ mối quan hệ giữa họ. Bạn có một thông tin quan trọng cần báo cho tất cả N lập trình viên này biết. Viết chương trình chỉ ra một số ít nhất các lập trình viên mà bạn cần cho họ biết thông tin, sao cho những người đó có thể thông báo cho tất cả những người còn lại thông tin của bạn.

Yêu cầu:

Dữ liệu đầu vào của chương trình dưới dạng file text có cấu trúc như sau:

+ Dòng đầu tiên chứa số lập trình viên N ($N < 500$)

+ Dòng thứ i trong N dòng tiếp theo chứa danh sách các lập trình viên mà người thứ i biết địa chỉ email của họ.

Dữ liệu đầu ra của dưới dạng file text có cấu trúc như sau:

+ Dòng đầu ghi số người ít nhất cần cho họ biết thông tin

+ Dòng thứ 2 ghi chỉ số của những người đó.

Ví dụ:

FileInput.inp	FileOutput.out
6	3
2 3	1 4 6
1	
5	
4	
1	